

# *FastTT*: Accelerating Shift-XOR Erasure Coding for Data Storage

Duo Sun   Ximing Fu   Qiang Wang

Harbin Institute of Technology (Shenzhen)  
Pengcheng Laboratory

IPDPS 2026

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

Duo Sun Ximing Fu Qiang Wang

Harbin Institute of Technology (Shenzhen)  
Pengcheng Laboratory

IPDPS 2026

Good afternoon. I am Duo Sun, presenting FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage, joint work with my supervisors Prof. Ximing Fu and Prof. Qiang Wang at HIT Shenzhen. Two-tone is a Shift-XOR erasure code that replaces finite-field multiplications with bit-shifts and XORs. This talk is about turning that theoretical efficiency into a practical, high-throughput implementation inside Ceph.

# Motivation

- ▶ Erasure coding = reliability + low storage cost.
- ▶ **Reed-Solomon (RS) Codes:** mature; many system-level optimizations already exist.
- ▶ **Two-tone Shift-XOR:** excellent theoretical complexity — only bit-shifts & XORs, no finite-field multiplications.
  - ▶ But a *direct* implementation is **memory-bound**.
  - ▶ RS optimizations do **not** transfer — different compute & memory-access patterns.

## Our Goal

First high-performance, memory-access-aware implementation of Two-tone Shift-XOR, validated in Ceph.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

## Motivation

### Why This Problem Matters

#### Motivation

- ▶ Erasure coding  $\Rightarrow$  reliability + low storage cost.
- ▶ **Reed-Solomon (RS) Codes**: mature; many system-level optimizations already exist.
- ▶ **Two-tone Shift-XOR**: excellent theoretical complexity — only bit-shifts & XORs, no finite-field multiplications.
  - ▶ But a direct implementation is **memory-bound**.
  - ▶ RS optimizations do **not** transfer — different compute & memory-access patterns.

#### Our Goal

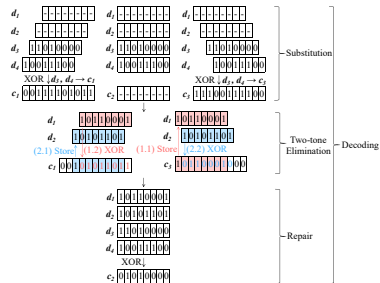
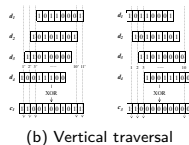
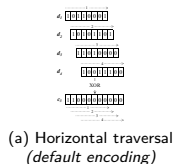
First high-performance, memory-access-aware implementation of Two-tone Shift-XOR, validated in Ceph.

Erasure coding delivers reliability at much lower storage cost than replication. RS Codes are the dominant choice and already heavily optimized. Our focus is Two-tone Shift-XOR: theoretically excellent, only shifts and XORs, no finite-field multiplications. The gap is on the implementation side: a direct Two-tone implementation is memory-bound, and RS-specific tricks do not transfer because the computation and memory-access patterns are fundamentally different. FastTT is the first high-performance, memory-access-aware implementation, validated in Ceph.

# Limitations of Existing Solutions

## Encoding & Decoding

- ▶ Algorithm & correctness established in prior work — briefly recalled here.
- ▶ Two traversal patterns of the data-parity grid: *horizontal* (default direct implementation) and *vertical*.
- ▶ **Decoding:** generic 3-stage flow — Substitution → Two-tone Elimination → Repair (see fig. (c)).



**Bottleneck = memory access, not arithmetic**

**Both encoding & decoding:** poor locality, repeated access of the same chunks.

**Decoding extra:** recovering lost *data* and re-encoding lost *parity* revisit overlapping memory.

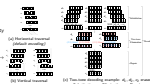
# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

- Limitations of Existing Solutions
  - Encoding & Decoding Bottlenecks

## Limitations of Existing Solutions

### Encoding & Decoding

- Algorithms & correctness established in prior work — barely revisited here
- Two traversal patterns of the data-parity grid: horizontal (default direct implementation) and vertical
- Decoding: generic 3-stage flow —  
 Substitution → Two-tone Elimination → Repair (see fig. (c))



Bottleneck = memory access, not arithmetic

Both encoding & decoding: poor locality, repeated access of the same chunks.

Decoding extra: re-encoding lost data and re-encoding lost parity revisit overlapping memory.

The Two-tone encoding and decoding algorithms, together with their correctness, were established in prior work, so I will skip the math and focus on the implementation gap. For encoding, horizontal traversal of the data-parity grid is the default direct implementation, with vertical as the alternative. Decoding follows a generic three-stage flow — substitute survived data, run Two-tone elimination, then repair the lost parity — illustrated on the right with  $d_1$ ,  $d_2$  and  $c_2$  erased. The bottleneck is the same on both sides: poor locality and repeated access over the same chunks. Decoding pays an extra cost: solving for lost data and re-encoding lost parity are separate passes that revisit overlapping memory. The root cause is memory access behavior, not coding arithmetic.

# FastTT Overview

## Design principle

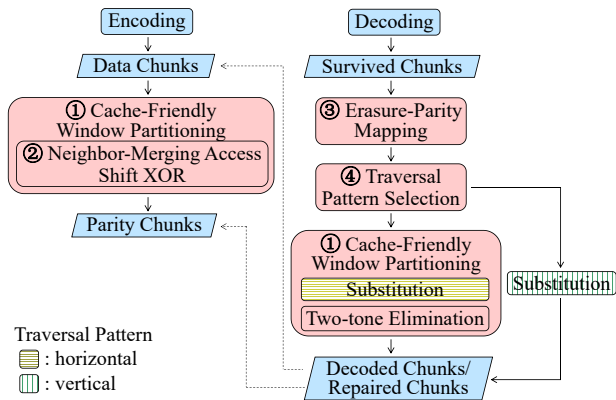
Optimize Two-tone by **memory-access behavior**, not by algorithmic rewriting.

### Four coordinated techniques

1. **CFWP** – Cache-Friendly Window Partitioning
2. **NMA** – Neighbor-Merging Access
3. **EPM** – Erasure-Parity Mapping
4. **TPS** – Traversal Pattern Selection

Encoding: CFWP + NMA.

Decoding: CFWP + NMA + EPM + TPS.



Overall *FastTT* workflow.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

- Approach and Key Ideas
- FastTT Overview

## Design principle

Optimize Two-tone by **memory-access behavior**, not by algorithmic rewriting.

### Four coordinated techniques

1. CFWP – Cache-Friendly Window Partitioning
2. NMA – Neighbor-Merging Access
3. EPM – Erasure-Parity Mapping
4. TPS – Traversal Pattern Selection

Encoding: CFWP + NMA.

Decoding: CFWP + NMA + EPM + TPS.



FastTT follows one design principle. Instead of rewriting the Two-tone code construction, we optimize its memory-access behavior. Four coordinated techniques deliver this. Encoding uses Cache-Friendly Window Partitioning together with Neighbor-Merging Access to keep the working set in cache and cut redundant parity writes. Decoding reuses the same two and adds Erasure-Parity Mapping, which fuses data recovery with parity re-encoding into a single pass, and Traversal Pattern Selection, which picks the best access order for the erasure pattern at hand. The next four slides walk through these techniques one by one.



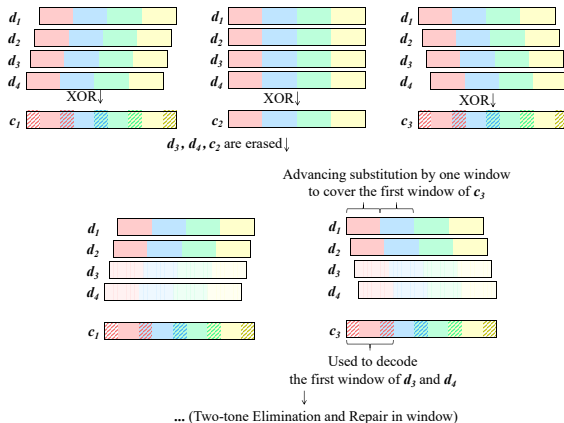
# Key Idea 1 – Cache-Friendly Window Partitioning

- ▶ Tile each chunk into **cache-sized windows**.
- ▶ Finish all Shift-XOR updates in one window before moving on.
- ▶ **On-demand init** – no full zero pass up front.

## Effect

Localized access + one fewer full traversal.

Benefits both encoding and decoding.



Each chunk is tiled into windows; all parity updates inside a window finish before advancing. Colored segments show local access.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

## Approach and Key Ideas

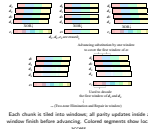
### Cache-Friendly Window Partitioning

#### Key Idea 1 – Cache-Friendly Window Partitioning

- Tile each chunk into **cache-sized windows**.
- Finish all Shift-XOR updates in one window before moving on.
- **On-demand init** – no full zero pass up front.

#### Effect

Localized access → one fewer full traversal.  
Benefits both encoding and decoding.



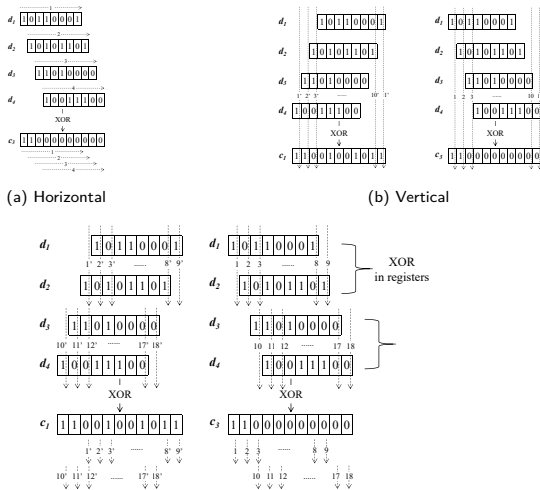
Throughout the key ideas I will use the (4,3) code as a running example, matching the figures. The idea of Cache-Friendly Window Partitioning is to tile each chunk into windows that fit in cache, and finish all Shift-XOR updates inside a window before moving to the next. The working set stays small and fits in cache. On-demand initialization also removes one full zero-pass over the chunk. This benefits both encoding and decoding.

## Key Idea 2 – Neighbor-Merging Access

- ▶ XOR **two adjacent** data symbols in SIMD registers *first*.
- ▶ Then write the merged result to parity *once*.
- ▶ Keeps horizontal-style read locality; halves parity writes.

### Effect

Fewer redundant accesses than *both* horizontal and vertical traversals in practice.



# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

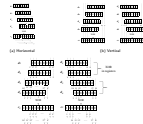
- Approach and Key Ideas
  - Neighbor-Merging Access

## Key Idea 2 – Neighbor-Merging Access

- XOR two adjacent data symbols in SIMD registers first.
- Then write the merged result to parity once.
- Keeps horizontal-style read locality; halves parity writes.

### Effect

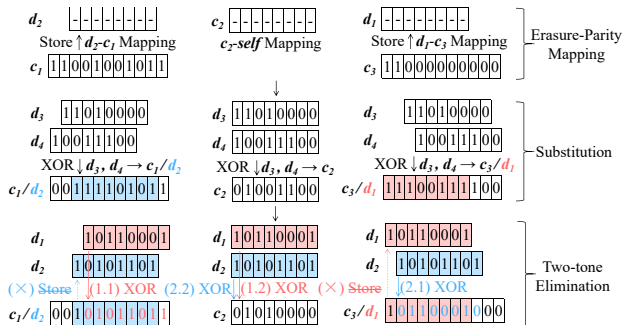
Fewer redundant accesses than both horizontal and vertical traversals in practice.



In the example, two adjacent data chunks share the same offset to a particular parity, so each contributes to that parity slot without any shift. Instead of writing to that parity slot twice, we XOR the two data symbols together inside a SIMD register first and then write the merged result once. This halves the parity writes for those positions while keeping the read pattern localized like the horizontal traversal. As a result, Neighbor-Merging incurs fewer redundant memory accesses than both the horizontal and the vertical traversals in practice.

# Key Idea 3 – Erasure-Parity Mapping

- ▶ Map each erased chunk (data or parity) to a survived parity for joint recovery.
- ▶ Decoded data symbols are **immediately** used to update erased parity — no separate repair pass.
- ▶ One unified decoding flow for *all* erasure patterns.



## Effect

Naive: 3 passes (substitute  $\rightarrow$  eliminate  $\rightarrow$  **repair**).

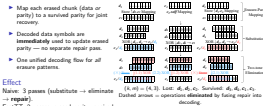
FastTT: 2 passes; erased parity repaired *along the way*.

$(k, m) = (4, 3)$ . Lost:  $d_1, d_2, c_2$ . Survived:  $d_3, d_4, c_1, c_3$ .  
Dashed arrows = operations **eliminated** by fusing repair into decoding.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

- Approach and Key Ideas
  - Erasure-Parity Mapping

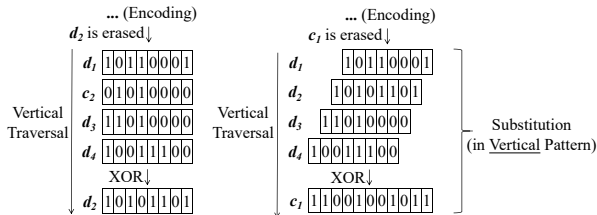
## Key Idea 3 – Erasure-Parity Mapping



In the  $(4,3)$  example with  $d_1$ ,  $d_2$ ,  $c_2$  lost: naive decoding substitutes  $d_3$ ,  $d_4$  into  $c_1$ ,  $c_3$ , runs elimination to recover  $d_1$  and  $d_2$ , then performs a separate third pass to re-encode  $c_2$  from scratch. That third pass re-reads all data, which is redundant. FastTT maps each erased chunk to a survived parity:  $d_1 \rightarrow c_1$ ,  $d_2 \rightarrow c_3$ ,  $c_2 \rightarrow c_2$  itself. During elimination, every decoded symbol of  $d_1$  or  $d_2$  is immediately XORed into the  $c_2$  buffer. By the time elimination finishes,  $c_2$  is already repaired. The dashed arrows in the figure show the operations eliminated by this fusion.

## Key Idea 4 – Traversal Pattern Selection

- ▶ Only **1 chunk** lost  $\Rightarrow$  each survived chunk read **once**, result written **once**.
- ▶ Vertical pattern:  $\Theta((k+1)l)$ .  
Horizontal:  $\Theta((k+m)l)$  — extra parity reads/writes for the  $m-1$  untouched parities.
- ▶ No merging needed: each data chunk contributes exactly once, so pairwise XOR adds no benefit.
- ▶ All accesses are **sequential & single-pass**  $\Rightarrow$  use SIMD **streaming stores** (bypass cache, avoid pollution).



$(k, m) = (4, 3)$ . Lost:  $d_2$  only.  
Reconstructed by one XOR pass over  $d_1, d_3, d_4$  with shifted offsets from  $c_1$ .

No elimination, no merging — just stream-read, XOR, stream-write.

### Single-erasure vs. ISA-L

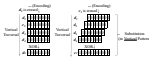
+135% – +182% improvement  
(i.e., FastTT is  $2.35\times$  –  $2.82\times$  the speed of ISA-L)

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

- Approach and Key Ideas
  - Traversal Pattern Selection

## Key Idea 4 – Traversal Pattern Selection

- Only 1 chunk lost  $\Rightarrow$  each survived chunk read **once**, result written **once**.
- Vertical pattern:  $\Theta((k+1)l)$ .  
Horizontal:  $\Theta((k+m)l)$  — extra parity reads/writes for the  $m-1$  untouched parities.
- No merging needed: each data chunk contributes exactly once, so pairwise XOR adds no benefit.
- All accesses are **sequential & single-pass**  $\Rightarrow$  use SIMD streaming stores (bypass cache, avoid pollution).



### Single-erasure vs. ISA-L

+135% — +182% improvement  
 (i.e., FastTT is 2.35x–2.82x the speed of ISA-L)

When only one chunk is lost, each survived chunk is read exactly once and the result is written once. Vertical pattern costs  $\Theta((k+1)l)$ ; horizontal would still touch the  $m-1$  untouched parities, costing  $\Theta((k+m)l)$ . There is also no benefit from neighbor-merging: each data chunk contributes exactly one XOR, nothing to merge. Because all accesses are sequential and single-pass, we use SIMD streaming stores to bypass the cache entirely, avoiding pollution and maximizing bandwidth. In the (4,3) example with only  $d_2$  lost, we stream-read  $d_1$ ,  $d_3$ ,  $d_4$  with their shifted offsets from  $c_1$ , XOR them, and stream-write directly to  $d_2$ . No elimination needed. Result: single-erasure decoding is 135–182% faster than ISA-L, i.e.,  $2.35\text{--}2.82\times$  its speed. Multi-erasure falls back to EPM.



# Flagship Results – Throughput in Ceph

## vs. Ceph production baselines

Intel i9-14900K · AVX2 · 2 MB chunks

- ▶ vs. ISA-L: +59.1% encode, +50.9% decode
- ▶ vs. Jerasure: +261% encode, +161% decode

| $(k, m)$ | Encode (GB/s) |       |       | Avg. decode |       |       |
|----------|---------------|-------|-------|-------------|-------|-------|
|          | FastTT        | ISA-L | Jera. | FastTT      | ISA-L | Jera. |
| (6, 2)   | 34.7          | 24.4  | 17.0  | 60.8        | 31.6  | 29.5  |
| (6, 3)   | 31.7          | 17.6  | 7.6   | 46.2        | 27.5  | 20.7  |
| (8, 3)   | 31.4          | 17.0  | 7.1   | 46.9        | 24.9  | 17.7  |
| (10, 4)  | 17.6          | 13.6  | 4.6   | 32.2        | 21.9  | 13.5  |

## vs. academic SOTA (Cerasure)

- ▶ +13.2% encode on average
- ▶ +16.0% decode on average; peak +43.6%
- ▶ **Single-erasure: +135% – +182%** vs. ISA-L ( $2.35\times - 2.82\times$  the speed).

| $(k, m)$ | Encode (GB/s) |          | Avg. decode |          |
|----------|---------------|----------|-------------|----------|
|          | FastTT        | Cerasure | FastTT      | Cerasure |
| (6, 2)   | 48.0          | 44.2     | 67.6        | 59.7     |
| (6, 3)   | 32.0          | 31.0     | 49.4        | 44.8     |
| (8, 3)   | 32.8          | 28.7     | 48.7        | 43.9     |
| (10, 4)  | 24.6          | 19.4     | 37.3        | 33.4     |

Avg. decode = averaged over erasure counts  $1..m$ , for both data- and parity-chunk losses.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

## Flagship Results

## Industrial and Academic Baselines

### Flagship Results – Throughput in Ceph

#### vs. Ceph production baselines

Intel® Xeon® - A002 - 2MB cache

- vs. ISA-L: +58.1% encode, +50.0% decode
- vs. Jerasure: +261% encode, +161% decode

#### vs. academic SOTA (Cerasure)

- +13.2% encode on average
- +16.0% decode on average, peak +43.6%
- Single-erasure: +135% ~ +182% vs. ISA-L (2.35x ~ 2.82x the speed)

| (k, n)  | Encode (GB/s) |       |      | Avg. decode |       |      | Encode (GB/s) |          |        | Avg. decode |        |          |
|---------|---------------|-------|------|-------------|-------|------|---------------|----------|--------|-------------|--------|----------|
|         | FastTT        | ISA-L | Jera | FastTT      | ISA-L | Jera | FastTT        | Cerasure | FastTT | Cerasure    | FastTT | Cerasure |
| (8, 2)  | 38.7          | 24.6  | 17.2 | 66.2        | 20.8  | 26.9 | (8, 2)        | 48.0     | 44.2   | 57.6        | 59.7   | 59.7     |
| (8, 4)  | 32.7          | 17.8  | 7.8  | 48.2        | 27.5  | 20.7 | (8, 4)        | 32.0     | 31.0   | 49.4        | 44.8   | 44.8     |
| (8, 8)  | 31.4          | 17.0  | 7.1  | 46.8        | 28.8  | 17.7 | (8, 8)        | 32.8     | 28.7   | 48.7        | 43.9   | 43.9     |
| (16, 4) | 17.8          | 10.8  | 4.8  | 32.2        | 21.9  | 13.9 | (16, 4)       | 24.6     | 19.4   | 37.3        | 33.4   | 33.4     |

Avg. decode = averaged over erasure counts 1..n for both data and parity chunk sizes.

All three baselines — ISA-L, Jerasure, and Cerasure — are RS-based libraries. Shift-XOR codes have no prior production-grade implementations in storage systems, so this is also the first time Two-tone is benchmarked head-to-head against optimized RS implementations under the same Ceph framework. Encoding-wise, FastTT consistently outperforms all baselines, including the academic state-of-the-art Cerasure, confirming that our memory-aware design effectively translates Two-tone's theoretical advantage into practical throughput.

For decoding, the average improvement is largely driven by the single-erasure case, where the TPS fast path uses streaming stores to deliver  $2.35\text{--}2.82\times$  the speed of ISA-L. As the number of erasures grows, the gap narrows: the single-erasure fast path no longer applies, and neighbor-merging cannot exploit the erased chunks during substitution because their content is unknown. Even so, FastTT still leads or ties the baselines on average across all configurations.

## Flagship Results – Ablation on (8, 3)

- ▶ **EPM**: first jump in *decode* (7.5  $\rightarrow$  16.9, +100%) – fusion matters.
- ▶ **CFWP**: lifts both encode & decode (+77% enc, +43% dec).
- ▶ **NMA**: pure encoding gain (23.3  $\rightarrow$  31.2, +34%).
- ▶ **TPS**: second decode jump (24.1  $\rightarrow$  35.2, +46%).

| Cache-Friendly Window Part. (CFWP) | Neighbor-Merging Access (NMA) | Erasure-Parity Mapping (EPM) | Traversal Pattern Sel. (TPS) | Encode      | Decode      |
|------------------------------------|-------------------------------|------------------------------|------------------------------|-------------|-------------|
|                                    |                               |                              |                              | 13.2        | 7.5         |
|                                    |                               | ✓                            |                              | 13.2        | 16.9        |
| ✓                                  |                               | ✓                            |                              | 23.3        | 24.1        |
| ✓                                  | ✓                             | ✓                            |                              | 31.2        | 24.1        |
| ✓                                  | ✓                             | ✓                            | ✓                            | <b>32.8</b> | <b>35.2</b> |

Throughput in GB/s; decode averaged over erasure counts 1..*m*.

### Takeaway

Each technique targets a distinct bottleneck; contributions are additive.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

## Flagship Results

## Ablation of the Four Techniques

### Flagship Results – Ablation on (8,3)

|                                                                                 | Cache-Friendly<br>Window Perm.<br>(CFWP) | Singleton<br>Storage Access<br>(NMA) | Erasure Parity<br>Repairing<br>(EPM) | Threshold<br>Parity Set<br>(TPS) | Encode | Decode |
|---------------------------------------------------------------------------------|------------------------------------------|--------------------------------------|--------------------------------------|----------------------------------|--------|--------|
| ► <b>EPM</b> : first jump in decode<br>(7.5 → 16.0, +100%) –<br>fusion matters. |                                          |                                      | ✓                                    |                                  | 11.2   | 7.6    |
| ► <b>CFWP</b> : lifts both encode &<br>decode<br>(+77% enc, +43% dec)           | ✓                                        |                                      | ✓                                    |                                  | 13.2   | 10.9   |
| ► <b>NMA</b> : pure encoding gain<br>(23.3 → 31.2, +34%)                        |                                          | ✓                                    | ✓                                    |                                  | 21.1   | 24.1   |
| ► <b>TPS</b> : second decode jump<br>(24.1 → 35.2, +46%)                        |                                          |                                      |                                      | ✓                                | 32.8   | 35.2   |

Throughput in GB/s; decode averaged over erasure counts 1..3n.

### Takeaway

Each technique targets a distinct bottleneck; contributions are additive.

The ablation on (8,3) decomposes where the gains come from. EPM alone more than doubles decoding throughput, which validates our claim that fusing data recovery with parity repair removes a real bottleneck — the redundant memory traffic of the separate repair stage. CFWP then improves both sides because cache-sized windows benefit any traversal, encoding or decoding. NMA is encoding-specific by design: it merges adjacent data contributions before writing parity, which only applies on the encoding side, so decoding stays flat here. Finally TPS gives a second large decoding jump by routing single-erasure cases to the streaming-store fast path. The four techniques target different bottlenecks, and the table shows their contributions are additive — each one matters.

# Summary and Future Research

## One-sentence summary

*FastTT* is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

## What to remember

- ▶ Bottleneck = **memory behavior**, not arithmetic.
- ▶ Four coordinated optimizations: **CFWP** + **NMA** (encode), **EPM** + **TPS** (decode).
- ▶ Single-erasure recovery benefits most – the **common** case in production.

## Open directions

- ▶ Adaptive / heterogeneous window sizing beyond fixed  $W$ .
- ▶ Multi-level SIMD-aware merging beyond pairwise.
- ▶ Multi-erasure recovery: decompose into reusable single-erasure steps.

# FastTT: Accelerating Shift-XOR Erasure Coding for Data Storage

- └ Summary and Future Research
  - └ Summary

## Summary and Future Research

### One-sentence summary

FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding, turning a theoretically efficient code family into a practical high-throughput storage primitive.

### What to remember

- ▶ Bottleneck = **memory behavior**, not arithmetic.
- ▶ Four coordinated optimizations: **CFWP** + **NMA** (encode), **EPM** + **TPS** (decode).
- ▶ Single-erasure recovery benefits most – the **common** case in production.

### Open directions

- ▶ Adaptive / heterogeneous window sizing beyond fixed  $W$ .
- ▶ Multi-level SIMD-aware merging beyond pairwise.
- ▶ Multi-erasure recovery: decompose into reusable single-erasure steps.

FastTT is the first high-performance systems implementation of Two-tone Shift-XOR coding. The bottleneck is memory behavior, not arithmetic. Four coordinated techniques: CFWP and NMA for encoding, EPM and TPS for decoding. Single-erasure recovery, the dominant case in production, benefits most. Open directions: adaptive window sizing, multi-level SIMD merging, and decomposing multi-erasure patterns into reusable single-erasure steps. Thank you.

# Thank you!

## Questions & Discussion

Duo Sun, Ximing Fu, Qiang Wang  
HIT (Shenzhen) · Pengcheng Laboratory

2026-05-16

# *FastTT*: Accelerating Shift-XOR Erasure Coding for Data Storage

- └ Summary and Future Research
- └ Thanks

Thank you!

Questions & Discussion

Duo Sun, Ximing Fu, Qiang Wang  
HIT (Shenzhen) - Pingcheng Laboratory

Thank you for listening. I am happy to discuss the implementation details, the memory-access modeling behind each of the four techniques, the comparison with Reed-Solomon libraries, or the implications for production storage deployments in Ceph.